

Solving AsteroidStatic with Deep Reinforcement Learning

Task 2 - Deep Reinforcement Learning (Prof. Thomas Nierhoff, OTH Amberg-Weiden)

Author: Himanshu Mohan Nikam

1. Introduction and Problem

This project trains reinforcement-learning agents to solve the provided AsteroidStatic environment. The agent is a point-mass spaceship in a 2D arena populated with 25 circular asteroids. It is driven by a continuous acceleration action and drifts due to inertia. An episode succeeds when the ship reaches within 0.30 of the goal, and fails on collision with an asteroid, on leaving the arena, or on reaching the 500-step time limit.

Observation: $obs = [x, y, vx, vy]$ (position and velocity only - the goal and asteroid positions are NOT observable). Action: $a = [ax, ay]$, each a real value in $[-1, 1]$. Reward (already shaped by the environment): $+0.5 \times$ progress toward the goal each step, $+10$ for reaching the goal, and -100 for a crash. The agent learns purely from observations, rewards and info - it may not read obstacle positions or plan against the geometry. All provided files are used unmodified, and only the random seed is varied to obtain different arenas.

Two graded objectives are measured, each averaged over ten arenas (seeds 0-9): (1) sample efficiency - the fewest environment transition calls until the goal is first reached; and (2) speed - the fewest time steps taken to reach the goal once trained.

2. Approaches

Because the environment is a continuous-control navigation task, we investigated two complementary families of algorithms and implemented a progression within each.

2.1 Value-based methods (DQN family)

We first implemented and tried the DQN approach. Deep Q-Networks learn a value $Q(s, a)$ over a finite set of actions, so the continuous action is discretized into 9 bang-bang actions - every combination of $\{-1, 0, +1\}$ for ax and ay . Full-throttle pushes are also the fastest way to move a mass, which helps the speed objective. Starting from vanilla DQN we added the standard improvements: Double DQN (DDQN), which decouples action selection from evaluation to reduce value over-estimation, and Dueling DDQN, which splits the network into a state-value stream $V(s)$ and an advantage stream $A(s, a)$. All three share experience replay, a target network, and epsilon-greedy exploration.

2.2 Policy-based methods (continuous control)

The second family acts directly on the continuous action, avoiding discretization. A deterministic actor $\mu(s)$ proposes an action and a critic $Q(s, a)$ evaluates it. We implemented a progression of three: a base Actor-Critic; DDPG, which stabilizes it with soft (Polyak) target updates and temporally-correlated Ornstein-Uhlenbeck exploration noise; and TD3, which adds twin critics with a clipped-minimum target (to curb over-estimation), target-policy smoothing, and delayed actor updates. Continuous control offers fine, graded control - including gentle braking - which the fixed full-throttle actions cannot express, and is therefore expected to produce shorter, smoother paths.

2.3 Methods considered

We also considered but did not focus on: Deep SARSA, an on-policy method that is naturally more risk-averse near the large -100 crash penalty; and Soft Actor-Critic (SAC) and PPO as alternative continuous-control algorithms with stronger, entropy-driven exploration. These are discussed as future directions.

3. Key Implementation Techniques

Several practical measures were decisive in getting the methods to learn in this obstacle-dense, sparse-goal environment:

- Input normalization. Positions and velocities are rescaled to a comparable range using the known environment bounds. This is a simple input scaling (not analysis of the environment) and was the single largest contributor to stable training.
- Reward handling. The -100 crash penalty dominates the small +0.5 progress-shaping term by roughly two orders of magnitude, which collapses the learned value estimates and leaves no usable signal to distinguish actions. We apply an asymmetric reward clip that floors the crash penalty while keeping the +10 goal bonus large and salient, so the rare successful transitions strongly influence learning. This deliberate reshaping is noted as a design choice.
- Exploration. The task is hard-exploration: a purely random agent reaches the goal in only about 0.2% of episodes and never at all on several arenas, because the goal lies behind dense obstacles at least three units away. We therefore keep exploration strong and persistent - full-magnitude random action injection for the value-based agents, and correlated Ornstein-Uhlenbeck noise for the continuous agents - so the agent keeps discovering the goal rather than collapsing into a trivial 'do nothing' or 'flee to safety' policy.
- Stability. Gradient clipping, target networks, and a replay warm-up period were used throughout.

4. Experimental Setup

All agents are implemented in PyTorch and use the environment's default settings; only the seed is changed to obtain different arenas. Hyperparameters are optimized with Optuna. Following the task rule, tuning is performed on seeds that are disjoint from the evaluation seeds: candidate seeds 100-119 were screened, and the two most tractable (seeds 115 and 119, on which the goal is at least occasionally reachable) were used for the search; the final numbers are reported on seeds 0-9. The tuning objective minimizes calls-to-first-solve, with a distance-based fallback so that configurations which do not solve within the episode budget are still ranked by how close they came. For each run we record calls-to-first-solve (Metric 1), the shortest successful episode length (Metric 2), and the calls associated with that shortest episode.

5. Results

The hyperparameter search and final evaluation are still in progress; the tables below will be populated once tuning completes. All values are averages over seeds 0-9.

Table 1. Method comparison on the two graded metrics.

Method	Metric 1 (calls)	Metric 2 (steps)	Assoc. calls
DQN	TBD	TBD	TBD
DDQN	TBD	TBD	TBD
Dueling DDQN	TBD	TBD	TBD
Actor-Critic	TBD	TBD	TBD
DDPG	TBD	TBD	TBD
TD3	TBD	TBD	TBD

Table 2. Tuned hyperparameters (best Optuna configuration per method).

Method	Best configuration
DQN family	TBD (to be filled from Optuna study)

Method	Best configuration
Actor-Critic	TBD (to be filled from Optuna study)
DDPG	TBD (to be filled from Optuna study)
TD3	TBD (to be filled from Optuna study)

Learning curves (episode length and success rate versus number of calls) and a trajectory plot of the trained policy overlaid on the arena will be added here to demonstrate that the arenas are solved.

6. Discussion

The dominant difficulty is exploration, not the learning rule. Because the goal sits behind a dense obstacle field, the straight-line progress reward can actively lure an agent into the asteroids just in front of the goal; states nearer the goal then carry higher crash risk and lower value, and an agent can rationally learn to flee to open space or, for the continuous deterministic policy, to output a near-zero 'do nothing' action and simply survive to the time limit. Overcoming this required exploration strong enough to actually reach the goal often enough for the value function to gain a positive anchor.

Two things that did not work are worth reporting. Symmetric reward clipping to $[-1, 1]$ left the crash penalty large enough to collapse all value estimates. And weak Gaussian exploration around an already-collapsed deterministic policy only jitters locally and never travels far enough to find the goal; strong, correlated or full-magnitude exploration was necessary. In our experiments the value-based agents, whose random exploration uses full-throttle actions, tended to discover the goal more readily, while the continuous agents produced smoother, shorter paths once they did learn.

7. Conclusion

We implemented and compared two families of deep RL agents on the AsteroidStatic task - a value-based DQN / DDQN / Dueling DDQN progression and a continuous-control Actor-Critic / DDPG / TD3 progression - together with the exploration and reward-shaping techniques needed to make them learn in this hard-exploration setting. The final averaged metrics and tuned configurations will be inserted once the Optuna searches and seed-0-9 evaluations complete; the stronger variants (Dueling DDQN and TD3) are expected to give the best sample efficiency and shortest paths respectively.